UNIT II – C++ DEBUGGING SOLUTIONS

Functions and Classes & Objects

Solutions to the five debugging problems in the uploaded Unit II question bank. Each solution includes defect analysis, corrected code, a dry run, and expected output.

Question 1: Dangling Reference in Return-by-Reference

Errors and root causes

- Line 7: Returning result by int& is a dangling-reference defect / undefined behavior. result is an automatic local variable; its lifetime ends when getMaximumScaled returns, so the caller receives a reference to an object that no longer exists.
- getArrayElement is safe for the shown use because arr refers to the caller-owned array. The returned element remains alive while the caller array exists.
- Correct approach: return the scaled maximum by value; retain reference return only for an existing caller-owned array element.

Corrected C++ code

```
#include <iostream>

int getMaximumScaled(int a, int b, int scaleFactor) {
    int maxVal = (a > b) ? a : b;
    return maxVal * scaleFactor;
}

int& getArrayElement(int arr[], int index) {
    return arr[index];
}

int main() {
    int x = 10, y = 20;
    int scaledMax = getMaximumScaled(x, y, 2);
    std::cout << "Scaled Max: " << scaledMax << '\n';

    int data[3] = {100, 200, 300};
    getArrayElement(data, 1) = 999;
    std::cout << "Updated Array[1]: " << data[1] << '\n';
    std::cout << "Scaled Max Value: " << scaledMax << '\n';
    return 0;
}
```

Dry run and expected output

max(10,20) = 20; scaled result = 20 × 2 = 40. Array element at index 1 is changed from 200 to 999. The saved value remains 40.

```
Scaled Max: 40
Updated Array[1]: 999
Scaled Max Value: 40
```

Question 2: Static Data Member Linkage and Member Function Scope

Errors and root causes

- Linker error: static data member totalBankReserves is declared inside the class but needs one definition at file scope (before C++17 inline-static alternatives). Without storage definition, the linker cannot resolve it.
- Compilation error: displayAudit() is static and has no this pointer. It can access static class data, but cannot directly access the per-object accHolder or balance. Use a non-static displayAccount() for account-specific details.

Corrected C++ code

```
#include <iostream>
#include <string>

class BankAccount {
private:
    std::string accHolder;
    double balance;
    static double totalBankReserves;

public:
    BankAccount(std::string name, double initialDeposit)
        : accHolder(name), balance(initialDeposit) {
        totalBankReserves += initialDeposit;
    }

    static void displayAudit() {
        std::cout << "Total Reserves in Vault: $"
                  << totalBankReserves << '\n';
    }

    void displayAccount() const {
        std::cout << "Account Holder: " << accHolder
                  << ", Balance: $" << balance << '\n';
    }
};

double BankAccount::totalBankReserves = 0.0;

int main() {
    BankAccount acc1("Alice", 5000.0);
    BankAccount acc2("Bob", 3000.0);

    BankAccount::displayAudit();
    acc1.displayAccount();
    acc2.displayAccount();
    return 0;
}
```

Dry run and expected output

Alice deposits 5000 and Bob deposits 3000. Aggregate reserves = 8000. Each account method prints its own holder and balance.

```
Total Reserves in Vault: $8000
Account Holder: Alice, Balance: $5000
```

```
Account Holder: Bob, Balance: $3000
```

Question 3: Function Overload Ambiguity via Default Arguments

Errors and root causes

- Call computeVolume(5.0) is ambiguous because the cuboid overload accepts one argument through default values and the cube overload directly accepts one argument. These candidate functions cannot be distinguished for this call.
- Call computeVolume(3.0, 7.0) is also ambiguous in the original code: the cuboid overload can accept two arguments (height defaults), and the cylinder overload accepts two arguments. Both are viable with identical parameter types.
- Refactor using distinct function names (volumeCuboid, volumeCube, volumeCylinder) so defaults remain useful without overload collisions.

Corrected C++ code

```
#include <iostream>

double volumeCuboid(double length, double width = 1.0,
                    double height = 1.0) {
    return length * width * height;
}

double volumeCube(double side) {
    return side * side * side;
}

double volumeCylinder(double radius, double height) {
    return 3.14159 * radius * radius * height;
}

int main() {
    std::cout << "Cube: " << volumeCube(5.0) << '\n';
    std::cout << "Cuboid: " << volumeCuboid(5.0) << '\n';
    std::cout << "Cylinder: " << volumeCylinder(3.0, 7.0) << '\n';
    return 0;
}
```

Dry run and expected output

Cube: $5^3 = 125$. Cuboid with omitted width and height: $5 \times 1 \times 1 = 5$. Cylinder: $\pi \times 3^2 \times 7 \approx 197.92017$.

```
Cube: 125
Cuboid: 5
Cylinder: 197.92
```

Question 4: Array of Objects Missing Default Constructor

Errors and root causes

- Student batch[3] requires three default-constructed Student objects. Since the class defines a parameterized constructor, the compiler does not implicitly generate a default constructor.

- Technique A: add an explicit default constructor. Technique B: initialize the array directly with three parameterized Student objects. The code below demonstrates Technique A.

Corrected C++ code

```
#include <iostream>
#include <string>

class Student {
private:
    int rollNumber;
    std::string name;

public:
    Student() : rollNumber(0), name("") {}

    Student(int r, std::string n)
        : rollNumber(r), name(n) {}

    void display() const {
        std::cout << "Roll: " << rollNumber
                  << ", Name: " << name << '\n';
    }
};

int main() {
    Student batch[3];

    batch[0] = Student(101, "Aarav");
    batch[1] = Student(102, "Bhavna");
    batch[2] = Student(103, "Chirag");

    for (int i = 0; i < 3; ++i) {
        batch[i].display();
    }
    return 0;
}
```

Dry run and expected output

The array is first default-constructed, then each slot is assigned a parameterized Student temporary. The loop displays the three records.

```
Roll: 101, Name: Aarav
Roll: 102, Name: Bhavna
Roll: 103, Name: Chirag
```

Question 5: Friend Function Declaration and Scope Resolution

Errors and root causes

- Forward declaration: TelemetryAuditor must be declared before it is named as a parameter in Engine's friend declaration. A class declaration alone is sufficient for a reference or value parameter declaration; the complete class is defined later.

- Inside inspectDiagnostics, alertThreshold belongs to the auditor object, so access it as a.alertThreshold. A friend function is a non-member and has no implicit this pointer.
- v8.inspectDiagnostics(v8, audit) is invalid because the friend is not a member. Invoke it as inspectDiagnostics(v8, audit).

Corrected C++ code

```
#include <iostream>

class TelemetryAuditor;

class Engine {
private:
    int temperature;
    int rpm;

public:
    Engine(int t, int r) : temperature(t), rpm(r) {}

    friend void inspectDiagnostics(Engine e, TelemetryAuditor a);
};

class TelemetryAuditor {
private:
    int alertThreshold;

public:
    TelemetryAuditor(int threshold) : alertThreshold(threshold) {}

    friend void inspectDiagnostics(Engine e, TelemetryAuditor a);
};

void inspectDiagnostics(Engine e, TelemetryAuditor a) {
    if (e.temperature > a.alertThreshold) {
        std::cout << "Warning: Engine overheat!" << '\n';
    } else {
        std::cout << "Engine operating nominally at "
                  << e.rpm << " RPM." << '\n';
    }
}

int main() {
    Engine v8(105, 3200);
    TelemetryAuditor audit(100);

    inspectDiagnostics(v8, audit);
    return 0;
}
```

Dry run and expected output

Engine temperature 105 exceeds auditor threshold 100, so the warning branch executes.

```
Warning: Engine overheat!
```

Quick Revision: Unit II Rules

- Never return a reference to a local automatic variable.
- Define static data members at file scope when required; static methods have no this pointer.
- Default arguments can make overloads ambiguous; choose non-overlapping signatures or distinct names.
- An object array requires a usable default constructor unless each element is directly initialized.
- Friend functions are non-members: pass objects explicitly and call them by function name, not object.member syntax.